Bhaskaracharya College of Applied Sciences
UNIVERSITY OF DELHI
Department of Computer Science

SOFTWARE ENGINEERING

SKILL MATCH

SEM V PROJECT

2026

TEAM

Ankit Kumar
2402009

Devajyoti mishra
2402016

Akanksha sundli
2402005

Sakshi
2402053

SKILL EXCHANGE
Software Engineering Project Report

Baseline academic documentation • Version 0.2 • 11 November 2026

Repository: https://github.com/securescript95/Skill-Match

Prepared in alignment with ISO/IEC/IEEE 29148:2018. Draft for academic review; not a certification or claim of independently verified full conformity.

Implementation status

The inspected repository contains a single interactive HTML/CSS/JavaScript prototype (Skill Exchange.html). Sample users, requests, chats and scheduling actions are held in browser memory. No persistent backend, production authentication, server-side authorization or real-time messaging was found during inspection. This report distinguishes prototype behavior from proposed target requirements.

Contents

• 1. Introduction and problem statement

• 2. Process model

• 3. Stakeholders and product context

• 4. Repository inspection

• 5. Software Requirements Specification

• 6. Use cases

• 7. Data dictionary

• 8. Design engineering

• 9. Project management and estimates

• 10. Coding and implementation status

• 11. Testing and basis path analysis

• 12. Acceptance criteria and traceability

• 13. ISO/IEC/IEEE 29148 alignment

• 14. Conclusion and references

Document ID: SE-PROJ-REP-001

Version: 0.2 (baseline draft)

Prepared: 11 November 2026

Repository: https://github.com/securescript95/Skill-Match

Status: Draft for academic review

Requirements basis: Prepared in alignment with ISO/IEC/IEEE 29148:2018. This is not a certification or a claim of independently verified full conformity.

Implementation-status statement: The inspected repository contains a single interactive HTML/CSS/JavaScript prototype (`Skill Exchange.html`). It demonstrates screens and simulated interactions. Sample users, requests, chats and scheduling actions are held in browser memory. No persistent backend, production authentication, server-side authorization, or real-time messaging was found during inspection. This report distinguishes prototype behavior from proposed requirements.

Document control

Item	Value
Project	Skill Exchange (repository: Skill-Match)
Product	Mobile-first responsive web application
Author	Student project team — enter actual names
Institution / course	To be completed
Faculty guide	To be completed
Version / date	0.1 / 11-Nov-2026
Approval	Pending stakeholder and faculty review

Executive summary

Skill Exchange is a proposed peer-to-peer learning platform for campus communities. A member publishes skills they can teach and skills they want to learn. The platform helps members discover compatible peers, express interest, review incoming requests, establish mutual matches, communicate, and arrange skill-exchange sessions.

The central value proposition is reciprocal learning: one participant's teachable skill can satisfy another participant's learning goal, ideally in both directions. The current frontend prototype demonstrates onboarding, skill selection, swipe-style discovery, incoming requests, a match screen, chat, scheduling, profile and settings views.

The proposed MVP evolves the prototype into a secure, persistent responsive web application. A modular monolith with Python FastAPI and PostgreSQL is recommended, subject to implementation validation. The MVP prioritizes profile and skill management, discovery, requests, mutual matching, match-authorized messaging and session proposals. AI recommendations, video calls, payments, calendar integrations and reputation systems are deferred.

1. Introduction

1.1 Problem statement

Students often want to acquire practical skills but may lack affordable, accessible, peer-level learning opportunities. At the same time, many students possess useful skills they could teach but have no structured way to find people who want to learn them. Informal campus groups can make discovery, coordination and follow-up difficult.

Skill Exchange addresses this coordination problem by representing both sides of a user's learning profile, helping users discover potential reciprocal partners, and providing a workflow to connect and arrange an exchange.

1.2 Objectives

• Let a member maintain a profile and list teachable and desired skills.

• Help members discover people with relevant skill compatibility.

• Separate outbound discovery decisions from incoming interest decisions.

• Create a match only after the required acceptance conditions are met.

• Provide private communication to matched participants.

• Let matched participants propose and confirm an exchange session.

• Provide a responsive interface usable on smartphones and desktop browsers.

• Apply privacy, access-control and safety rules to user-generated interactions.

1.3 Scope

MVP scope: registration/login; profile management; skill catalogue; TEACH and LEARN skill associations; Discover feed; pass/interest actions; separate Requests queue; accept/reject; mutual match; conversation inbox; persistent text messaging; exchange-session proposal and response; basic in-app notifications; block/report entry points; responsive interface and accessible non-swipe controls.

Out of scope for MVP: AI/ML recommendation models; integrated voice/video; payments; formal skill verification; public ratings/reputation; calendar-provider integrations; group learning; public social feeds; native mobile applications.

1.4 Terms

Term	Meaning
Member	Registered platform user
Teach skill	Skill a member offers to explain or teach
Learn skill	Skill a member wants to acquire
Interest	A member's expression of interest in another profile
Incoming request	Interest received by a member, awaiting their decision
Match	Mutually accepted connection between two members
Exchange session	Proposed or confirmed learning meeting
MVP	Minimum Viable Product
SRS	Software Requirements Specification
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
FP	Function Point
CFG	Control Flow Graph

2. Software process model

2.1 Selected model: iterative and incremental development

An iterative and incremental model is recommended. The prototype provides early feedback about the main user journey, while backend behavior and detailed requirements are still being refined. Short increments let the team validate core workflows before investing in advanced features.

Each iteration follows: refine requirements → design → implement → test → review → update documentation. Two-week iterations are a planning assumption, not a fixed commitment.

Increment	Outcome	Exit evidence
0 — Baseline	Scope, user journeys, SRS and prototype review	Requirements baseline reviewed
1 — Identity/profile	Registration, login, profile and skills	API/UI tests
2 — Discovery/requests	Discover, pass/interest, request decisions	State-transition tests
3 — Matching/chat	Mutual matches, inbox and messaging	Authorization/integration tests
4 — Sessions/safety	Session proposals, block/report, notifications	Acceptance tests
5 — Stabilization	Responsive, accessibility, security and deployment	Release checklist

3. Stakeholders and user classes

Stakeholder	Interest
Student member	Maintains profile, discovers peers, handles requests, chats and schedules sessions
Skill sharer / learner	Same member role; teaches and learns different skills
Moderator / administrator	Reviews reports and handles abuse; admin console may be future scope
Student developer	Designs, implements, tests and maintains software
Faculty supervisor	Reviews requirements, process, models, estimates, implementation and test evidence
External providers	May provide hosting, email, storage or notifications

The initial target population is campus students. Campus verification, age eligibility and whether registration is institution-restricted remain open decisions.

4. Product perspective and inspected prototype

The proposed system comprises a responsive browser client, Python API service, relational database, optional object storage for profile images, and optional notification provider.

Repository inspected: https://github.com/securescript95/Skill-Match

Root files observed: .gitignore, LICENSE, Skill Exchange.html.

The HTML file uses CSS variables, responsive viewport settings, vanilla JavaScript, sample profiles and in-memory state. It includes Welcome, Login, Signup, Teach, Learn, Discover, Requests, Chats, Chat detail, Match, Schedule, Profile and Settings screens.

Observed limitations:

• Login/signup are UI demonstrations; credentials are not verified.

• A right swipe immediately opens a simulated match screen and pre-populates a chat. This differs from the target pending-request → recipient-acceptance → mutual-match workflow.

• Requests only mutate a local array.

• Messages are held in browser memory; a simulated reply is appended by a timer.

• Scheduling uses sample text and does not persist/validate the selected form values.

• Profile and skill changes are local state only.

• No server-side authorization, database, notification provider or moderation service was found.

These are prototype limitations, not defects in a production backend.

5. Software Requirements Specification (SRS)

5.1 Standards alignment

This SRS is prepared in alignment with ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering. It uses uniquely identified, atomic and testable requirements; records assumptions and open decisions; and links requirements to verification evidence.

ISO describes the standard as specifying requirements-engineering processes and the content/format of requirements information items. The ISO page states that the 2018 edition was reviewed and confirmed in 2024 and notes a successor draft under development. This student report is not a licensed reproduction of the standard and does not claim certification or independently verified full conformity. A final clause-by-clause review requires access to the full standard and reviewer validation.

5.2 Requirement conventions

FR = Functional Requirement; NFR = Non-functional Requirement; BR = Business Rule. P0 = MVP must-have; P1 = important if time permits; P2 = deferred. Requirements describe the target system, not necessarily the current prototype.

5.3 Functional requirements

ID	Requirement	Priority	Verification
FR-001	The system shall allow a visitor to register with a unique email and password.	P0	Registration test
FR-002	The system shall allow a member to authenticate and log out.	P0	Login/logout test
FR-003	The system shall allow a member to view and update their own profile.	P0	Profile test
FR-004	The system shall provide searchable skill selection from a managed catalogue.	P0	Skill test
FR-005	The system shall associate a member with skills under TEACH or LEARN roles.	P0	Data/API test
FR-006	The system shall allow proficiency to be recorded for TEACH skills.	P1	Validation test
FR-007	Discover shall exclude the current member, blocked profiles and profiles already acted upon under the configured policy.	P0	Feed filtering test
FR-008	A member shall be able to PASS or EXPRESS_INTEREST in a discovered profile.	P0	Interaction test
FR-009	Incoming EXPRESS_INTEREST actions shall appear in a separate Requests view.	P0	Requests test
FR-010	The recipient shall be able to ACCEPT or REJECT a pending request.	P0	Request decision test
FR-011	The system shall create at most one active match for an unordered member pair after the acceptance rule is met.	P0	Uniqueness/integration test
FR-012	The system shall display active matches to their participants.	P0	Match list test
FR-013	The system shall provide a conversation for an active match.	P0	Conversation test
FR-014	Only active-match participants shall read or send messages in that conversation.	P0	Authorization test
FR-015	Text messages shall persist and be returned chronologically.	P0	Persistence test
FR-016	The inbox shall order conversations by most recent activity.	P0	Inbox test
FR-017	A matched member shall propose a session with topic, start time, duration and format.	P0	Session proposal test
FR-018	The other participant shall accept or decline a session proposal.	P0	Session response test
FR-019	Both participants shall see session details and status.	P0	Visibility test
FR-020	Blocking shall prevent further discovery and communication between the pair.	P0	Block test

ID	Requirement	Priority	Verification
FR-021	A member shall submit a report with category and optional details.	P1	Report test
FR-022	Primary workflows shall show clear empty, loading, success and error states.	P1	UI state test
FR-023	Pass and interest actions shall have button alternatives to swipe gestures.	P0	Keyboard/tap test
FR-024	A member shall be able to request account deletion and receive an explanation of the outcome.	P1	Deletion test
FR-025	The system shall provide in-app notifications for requests, matches, messages and session updates.	P1	Notification test

5.4 Non-functional requirements

Draft targets are proposed for a small academic MVP and must be validated against deployment resources.

ID	Requirement	Draft acceptance measure
NFR-001	Performance: ordinary API reads should complete within 2 seconds at the 95th percentile for 100 simulated concurrent users.	Load test
NFR-002	Availability target should be 99% monthly during evaluation, excluding planned maintenance.	Monitoring; validate target
NFR-003	Production traffic shall use HTTPS; secrets shall not be committed.	Configuration review
NFR-004	Passwords shall never be stored in plaintext; use an appropriate salted password-hashing scheme.	Storage inspection
NFR-005	Every user-owned/match-owned API object shall have server-side object-level authorization.	Negative tests
NFR-006	Members shall not retrieve other members' private account data or private messages without authorization.	Privacy tests
NFR-007	Core flows shall target WCAG 2.2 Level AA, including keyboard alternatives, focus visibility, labels and contrast.	Accessibility audit
NFR-008	A first-time test participant should complete onboarding and express interest without external instructions.	Usability test
NFR-009	Responsive web client shall support current stable versions of selected major browsers.	Browser matrix
NFR-010	Backend shall separate routing, validation, business logic and persistence.	Code review
NFR-011	Database constraints shall prevent orphaned records and duplicate active matches.	Constraint tests
NFR-012	Backups and a documented restore procedure shall exist before production use.	Restore rehearsal
NFR-013	API instances should be stateless where practical; business logic must not depend on browser memory.	Architecture review
NFR-014	Logs shall omit passwords and avoid unnecessary exposure of message contents.	Log review

Security design should consider OWASP API Security Top 10 (especially object-level authorization and broken authentication). Password handling should follow current NIST SP 800-63B guidance. Accessibility should be assessed against WCAG 2.2.

5.5 Business rules

• **BR-001 No self-interaction:** a member cannot pass or express interest in their own profile.

• **BR-002 Pair uniqueness:** duplicate actions must be idempotent or rejected without duplicate records.

• **BR-003 Incoming request:** when A expresses interest in B, B sees a pending request. A right swipe alone does not create a match.

• **BR-004 Match creation:** a match is created when B accepts A's pending request. If both independently express interest, the server reconciles to one match.

• **BR-005 Rejection:** rejection creates no match. Proposed MVP default: suppress a repeat request for a configurable cooling-off period; exact duration is open.

• **BR-006 Chat authorization:** only the two participants of an active match may access its conversation.

• **BR-007 Block:** blocking prevents new messages and new discovery/requests between the pair. Historical messages are retained under the approved safety/retention policy.

• **BR-008 End match:** ending a match disables new messages and session proposals; history becomes read-only subject to retention policy.

• **BR-009 Timezones:** store timestamps in UTC and retain an IANA timezone identifier for display. Show recipient-local time and label timezone when participants differ.

• **BR-010 Session lifecycle:** proposed → accepted or declined; accepted sessions may become completed or cancelled.

• **BR-011 Deletion:** deactivate identity and delete/anonymize personal data according to an approved retention schedule; limited legal/safety records may require retention.

5.6 Assumptions and dependencies

• Initial pilot is for a campus/community.

• Skill proficiency is self-declared; MVP does not certify skills.

• Server-backed features require internet connectivity.

• Email delivery may be needed for verification and recovery.

• FastAPI/PostgreSQL are proposed, not currently implemented.

• Skill catalogue and moderation policy need ownership.

• Hosting, domain, email, image storage and notification providers are not selected.

5.7 Open decisions

ID	Decision	Proposed MVP default
OD-01	Campus-only or public registration?	Campus pilot; optional institutional email rule
OD-02	Minimum age?	Validate with institution and applicable law
OD-03	Can rejected users request again?	Cooling-off period; duration TBD
OD-04	Can a user undo a pass?	No in MVP
OD-05	Does a match expire?	No automatic expiry
OD-06	Chat after unmatch/block?	Stop new messages; retain history per policy
OD-07	Retention period?	Obtain institutional/legal review
OD-08	In-person safety?	Recommend public campus venue; provide report/block
OD-09	Meeting link?	User-supplied link/details in MVP
OD-10	Default timezone?	Device/browser timezone, user-editable
OD-11	Display age?	Omit unless justified
OD-12	Backend/auth choice?	FastAPI + PostgreSQL; compare managed auth

6. Use cases

ID	Use case	Actor	Outcome
UC-01	Register	Visitor	Account created
UC-02	Sign in	Member	Authenticated session
UC-03	Manage profile	Member	Profile saved
UC-04	Manage skills	Member	TEACH/LEARN skills saved
UC-05	Discover profiles	Member	Eligible candidates displayed
UC-06	Pass/express interest	Member	Interaction recorded
UC-07	Review request	Member	Request accepted/rejected
UC-08	View match	Member	Match details available
UC-09	Send/read message	Matched member	Message persisted
UC-10	Propose session	Matched member	Session proposal created
UC-11	Respond to session	Matched member	Session accepted/declined
UC-12	Block/report	Member	Safety action recorded

ID	Use case	Actor	Outcome
UC-13	Request account deletion	Member	Deletion workflow initiated

UC-06 — Express interest

Preconditions: authenticated member; target is discoverable; target is not self or blocked.

Main flow: open Discover → view eligible profile and compatibility → choose interest by button or swipe → client sends target ID/action → server validates identity, eligibility and duplicate constraints → server stores interaction → target receives pending request → client receives confirmation and loads next profile.

Alternatives: invalid target returns safe error; duplicate request returns current state or conflict without duplication; network failure allows retry only with idempotency support.

Postcondition: one pending request exists; no match exists until recipient acceptance.

UC-07 — Accept request

Preconditions: authenticated member is recipient of a pending request.

Main flow: open Requests → select Accept → server verifies recipient ownership and pending state → server atomically updates request and creates one match → conversation becomes available → both participants see match.

Alternative: stale request returns current state without duplicate match; blocked pair cannot be matched.

7. Data dictionary and conceptual model

Entity	Key attributes	Constraints / purpose
User	user_id PK, email, password_hash, account_status, created_at	Unique email; never store plaintext password
Profile	profile_id PK, user_id FK unique, display_name, username, bio, photo_key, timezone	One profile per user
Skill	skill_id PK, name, category, is_active	Canonical skill catalogue
UserSkill	user_skill_id PK, user_id FK, skill_id FK, role, proficiency	role TEACH/LEARN; unique user/skill/role
Interaction	interaction_id PK, actor_id FK, target_id FK, action, status, created_at	PASS/INTEREST; actor ≠ target
Match	match_id PK, user_low_id FK, user_high_id FK, status, created_at	Canonical pair ordering; unique pair
Conversation	conversation_id PK, match_id FK unique, created_at	One conversation per match
Message	message_id PK, conversation_id FK, sender_id FK, body, created_at, read_at	Sender must be participant
ExchangeSession	session_id PK, match_id FK, proposer_id FK, topic, starts_at_utc, duration_minutes, timezone_id, format, status	Match participants only
Block	block_id PK, blocker_id FK, blocked_id FK, created_at	Unique ordered pair; no self-block
Report	report_id PK, reporter_id FK, reported_user_id FK, category, details, status	Restricted moderation access
Notification	notification_id PK, recipient_id FK, type, reference_id, read_at	In-app activity record

Integrity constraints: foreign keys; unique skill-role association; canonical unique match pair; no self-interaction; message sender participant validation; positive bounded duration; timezone-aware timestamps stored in UTC.

8. Design engineering

8.1 Proposed architecture

Use a modular monolith for the first release. One FastAPI deployment contains domain modules for authentication, profiles, skills, discovery, interactions, matching, chat, sessions, notifications and safety. This avoids premature microservice complexity while preserving module boundaries.

Proposed stack:

• Client: responsive web UI; Flutter can be considered if native apps become a committed requirement.

• API: Python + FastAPI.

• Database: PostgreSQL.

• ORM: SQLAlchemy or SQLModel (choose one).

• Migrations: Alembic.

• Tests: pytest and API integration tests.

• Chat: begin with authenticated HTTP message endpoints; add WebSockets after persistence and authorization are stable.

• Storage: private object storage for profile images.

8.2 Module responsibilities

Module	Responsibility
auth	Registration, login, session/token lifecycle
profiles	Profile read/update and visibility
skills	Catalogue and user-skill associations
discovery	Candidate filtering and compatibility explanation
interactions	Pass/interest and request state
matching	Atomic match creation and lifecycle
chat	Conversation access and message persistence
sessions	Session proposal, response and cancellation
safety	Block/report and enforcement
notifications	In-app event records and optional delivery

8.3 Reciprocal compatibility pseudocode

```
FUNCTION compatibility(userA, userB):
IF userA.id == userB.id:
RETURN invalid

aLearnsFromB = INTERSECTION(userA.learnSkills, userB.teachSkills)
bLearnsFromA = INTERSECTION(userB.learnSkills, userA.teachSkills)

IF EMPTY(aLearnsFromB) AND EMPTY(bLearnsFromA):
RETURN incompatible

reciprocal = NOT EMPTY(aLearnsFromB) AND NOT EMPTY(bLearnsFromA)

RETURN {
compatible: TRUE,
reciprocal: reciprocal,
skillsAlearns: aLearnsFromB,
skillsBlearns: bLearnsFromA
}
END FUNCTION
```

A compatibility score ranks candidates only; it must never create a match. Match creation requires explicit user consent.

9. Project management and estimation

All values are planning estimates, not measured actuals or a commercial quote.

9.1 Draft 10-week timeline

Week	Activity	Deliverable
1	Problem validation, scope, repository review	Scope baseline
2	SRS, use cases, DFD, data dictionary	Requirements package
3	UI refinement, ERD, architecture/API design	Design baseline
4	Environment, database, auth/profile	Increment 1
5	Skills and discovery	Increment 2
6	Interactions, requests and matching	Increment 3
7	Conversation and message authorization	Increment 4
8	Session scheduling and safety	Increment 5

Week	Activity	Deliverable
9	Integration, accessibility and security tests	Test evidence
10	Fixes, deployment, report and demo	Release candidate

9.2 Function Point estimate — provisional

Classroom planning approximation only; validate transaction counts and complexity before final submission.

Function type	Draft count	Assumed average weight	UFP
External Inputs (EI)	10	4	40
External Outputs (EO)	7	5	35
External Inquiries (EQ)	7	4	28
Internal Logical Files (ILF)	10	10	100
External Interface Files (EIF)	1	7	7
Total			**210 UFP**

With an illustrative VAF of 1.00 pending assessment, adjusted FP = 210. This is not an audited IFPUG count.

9.3 Effort and cost

Assumed productivity: 8 FP/person-month (draft). Implied effort = 210 / 8 = 26.25 person-months. A single student working 15 hours/week for 10 weeks contributes about 150 hours, or 0.94 person-month at 160 hours/month.

These figures reveal a scope/capacity mismatch: the full estimated system is larger than a 10-week part-time student MVP. Reduce the delivered scope, add team capacity or extend the schedule. Do not claim the entire 210-FP scope is complete.

Cost formula: Cost = effort hours × approved hourly rate + hosting/services + contingency. Monetary cost is TBD because no rate, team size or hosting choice has been supplied.

9.4 Risk register

ID	Risk	Probability	Impact	Mitigation
R-01	Scope exceeds semester capacity	High	High	Freeze MVP; defer advanced features
R-02	Prototype behavior differs from target rules	High	Medium	Validate flows and maintain traceability
R-03	Duplicate/inconsistent matches	Medium	High	Transaction and unique canonical pair constraint
R-04	Unauthorized chat access	Medium	High	Server-side participant checks and negative tests
R-05	Low campus adoption	Medium	High	Small pilot and seed skill catalogue
R-06	Harassment or in-person safety incident	Medium	High	Block/report, guidance and moderation process
R-07	Timezone errors	Medium	Medium	UTC + IANA zone; DST and cross-zone tests
R-08	Secret leakage	Medium	High	Environment variables, scanning and rotation
R-09	Provider outage	Medium	Medium	Document dependencies and graceful errors
R-10	Unclear retention/deletion obligations	Medium	High	Institutional/legal review before pilot

10. Coding and implementation status

The inspected prototype is a single HTML/CSS/JavaScript file with simulated data. A small standalone Python compatibility module is included in this package at backend/skill_matching.py, with tests in tests/test_skill_matching.py. It demonstrates reciprocal skill overlap only and is not connected to the HTML client, FastAPI or a database.

11. Testing and basis path analysis

Test levels: unit tests; API tests; database/integration tests; UI and browser tests; security tests; acceptance tests.

For the sample compatibility function, there are three decision points: same-user check, no-overlap check, and reciprocal-overlap check. Cyclomatic complexity:

V(G) = D + 1 = 3 + 1 = 4

Independent paths:

• Same user → invalid.

• Different users, no overlap either direction → incompatible.

• Different users, one-way overlap → compatible, non-reciprocal.

• Different users, overlap both ways → compatible, reciprocal.

The CFG, DFDs, ERD and sequence diagrams are included in Appendix A of this report. This is a basis-path test design for the sample module, not evidence of complete system testing.

12. MVP acceptance criteria

• Member can register, sign in and update a profile.

• Member can add at least one TEACH and one LEARN skill.

• Discover excludes self and blocked profiles.

• Interest creates a pending request, not an immediate match.

• Recipient can accept or reject a pending request.

• Acceptance creates exactly one match and a conversation.

• Only match participants can read/send messages.

• Messages persist after refresh and appear chronologically.

• Matched member can propose a session; the other can accept/decline.

• Core interactions have keyboard/tap alternatives and clear error feedback.

• Critical authorization and state-transition tests pass.

• Documentation reflects delivered implementation and known limitations.

13. Requirements traceability matrix

Goal	Requirements	Verification
G-01 Profile and skills	FR-001–FR-006	Registration/profile/skill tests
G-02 Discover peers	FR-007, FR-008, FR-023	Filtering and UI tests
G-03 User-controlled matching	FR-009–FR-012	Request lifecycle/uniqueness tests
G-04 Private communication	FR-013–FR-016, NFR-005, NFR-006	Authorization/persistence tests
G-05 Coordinate exchanges	FR-017–FR-019, BR-009, BR-010	Session/timezone tests
G-06 Safety and trust	FR-020–FR-024, NFR-003–NFR-008	Block/report/privacy/accessibility tests
G-07 Maintainability	NFR-010–NFR-014	Architecture/code/operations review

14. ISO/IEC/IEEE 29148 alignment checklist

Review topic	Evidence	Status
Purpose and scope	Sections 1.1–1.3	Drafted
Product context	Sections 3–4	Drafted
Stakeholders/user classes	Section 3	Drafted
Functional requirements	Section 5.3	Drafted; validation pending
Non-functional requirements	Section 5.4	Drafted; targets pending validation
Assumptions/dependencies	Section 5.6	Drafted
IDs/priorities	Section 5.2–5.4	Drafted
Verification/acceptance	Sections 5, 11, 12	Drafted
Traceability	Section 13	Initial matrix drafted

Review topic	Evidence	Status
Consistency/completeness review	Cross-section review	Pending
Stakeholder validation	No formal review recorded	Pending
Normative clause-by-clause audit	Requires full standard and reviewer	Not claimed

15. Conclusion and next steps

The repository demonstrates the intended interaction concept but is not yet a full-stack system. Immediate priorities are to validate business rules, approve the MVP boundary, finalize the data model, and implement request/match lifecycle with database-enforced uniqueness and server-side authorization.

• Fill in student, institution, course and supervisor details.

• Review OD-01 to OD-12 with the faculty guide and intended users.

• Approve the SRS baseline and acceptance criteria.

• Finalize DFD/ERD/API contracts.

• Integrate the Python sample module into FastAPI.

• Replace simulated client state with persistent API-backed workflows.

• Recalculate FP, effort and schedule after scope approval.

References

• ISO, **ISO/IEC/IEEE 29148:2018 — Systems and software engineering: Life cycle processes — Requirements engineering**: https://www.iso.org/standard/72089.html

• IEEE Standards Association, **ISO/IEC/IEEE 29148-2018**: https://standards.ieee.org/ieee/29148/6937/

• OWASP, **API Security Top 10 — 2023**: https://owasp.org/API-Security/

• W3C, **Web Content Accessibility Guidelines (WCAG) 2.2**: https://www.w3.org/TR/WCAG22/

• NIST, **SP 800-63B, Digital Identity Guidelines: Authentication and Authenticator Management**: https://pages.nist.gov/800-63-4/sp800-63b.html

• FastAPI documentation: https://fastapi.tiangolo.com/

• PostgreSQL documentation: https://www.postgresql.org/docs/

Review note: Add institution-specific cover-page details, actual screenshots, measured test results, real schedule and implementation evidence before final submission. Never report proposed features or draft estimates as completed or measured facts.

Appendix A — System diagrams

The diagrams below are embedded in this document so the report can be reviewed and shared as one editable Word file. They represent the proposed target system and its sample compatibility logic; they do not imply that these backend components are already implemented in the inspected prototype.

Appendix A — System diagrams

All diagrams below are integrated into this report and describe the proposed target MVP. They are design artifacts, not evidence that the backend has already been implemented.

A1. Context Data Flow Diagram (DFD Level 0)

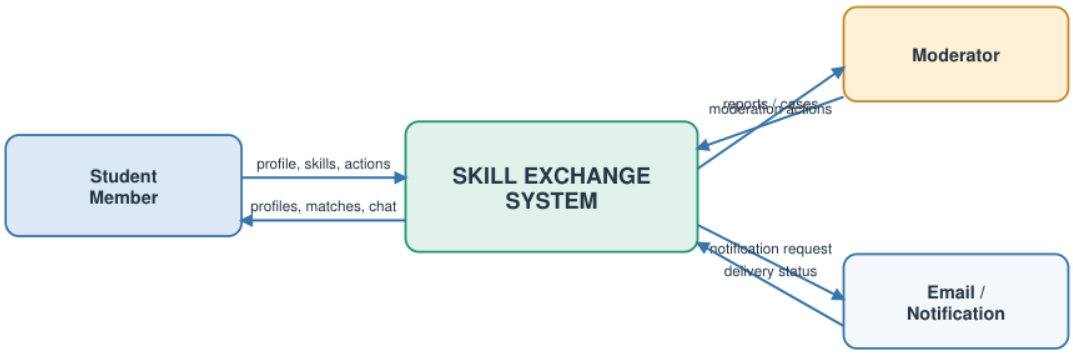


Figure A1. External actors and high-level information exchange.

A2. Data Flow Diagram (DFD Level 1)

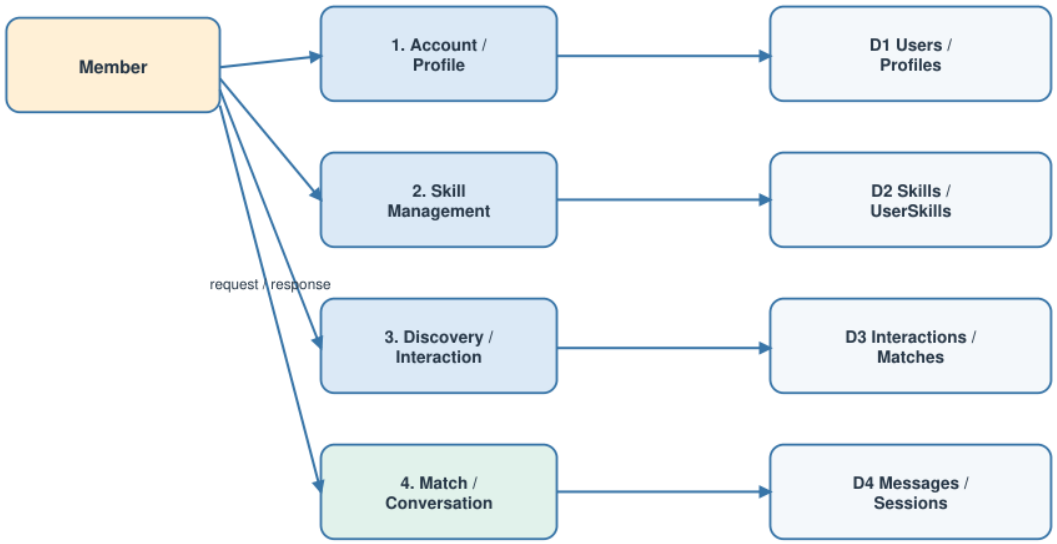


Figure A2. Main logical processes and data stores.

Appendix A — Diagram sheet 1

A3. Use-case diagram

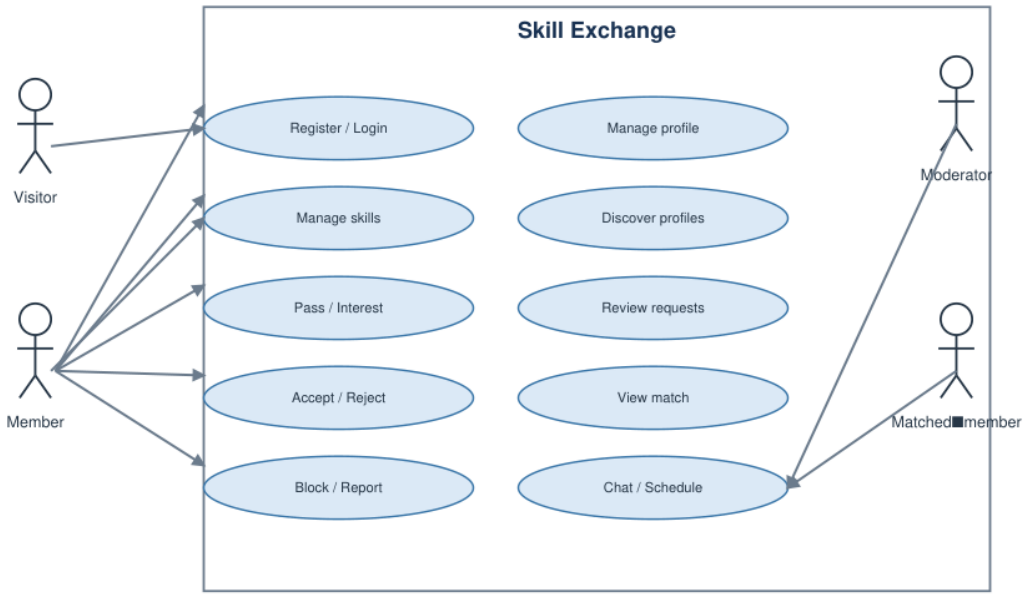


Figure A3. Primary actors and system use cases.

A4. Sequence diagram — express interest and accept request

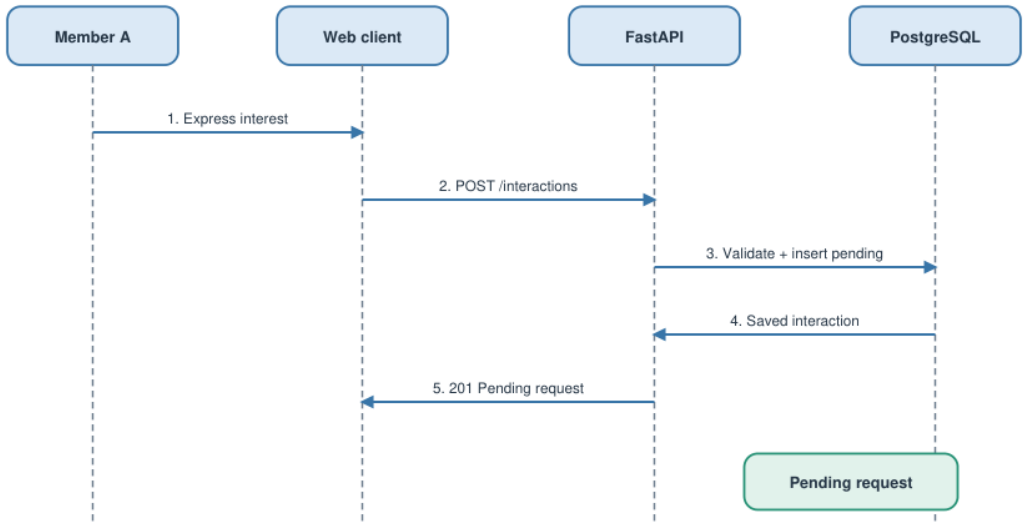


Figure A4. Interest is recorded as pending; recipient acceptance triggers match creation.

Appendix A — Diagram sheet 2

A5. Sequence diagram — authorized message flow

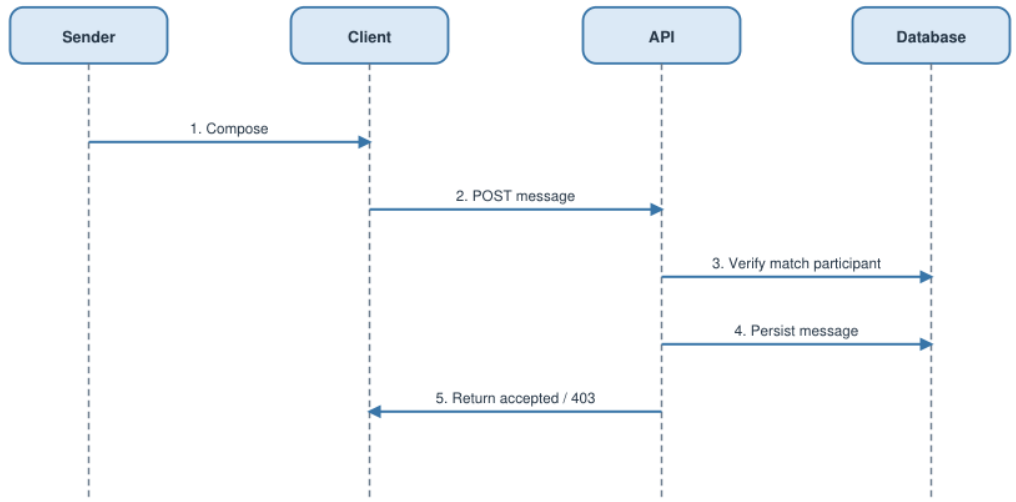


Figure A5. Server verifies match participation before persisting a message.

A6. Conceptual Entity Relationship Diagram (ERD)

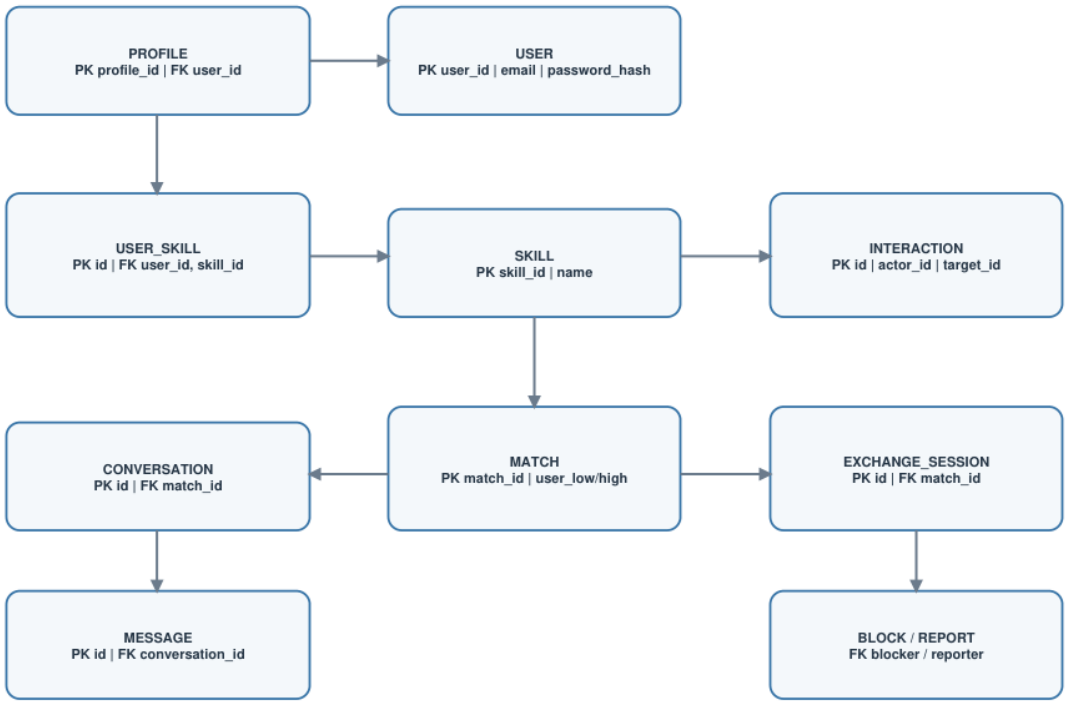


Figure A6. Core entities and principal relationships. Foreign-key and uniqueness constraints are detailed in Section 7.

Appendix A — Diagram sheet 3

A7. Control Flow Graph (CFG) — compatibility function

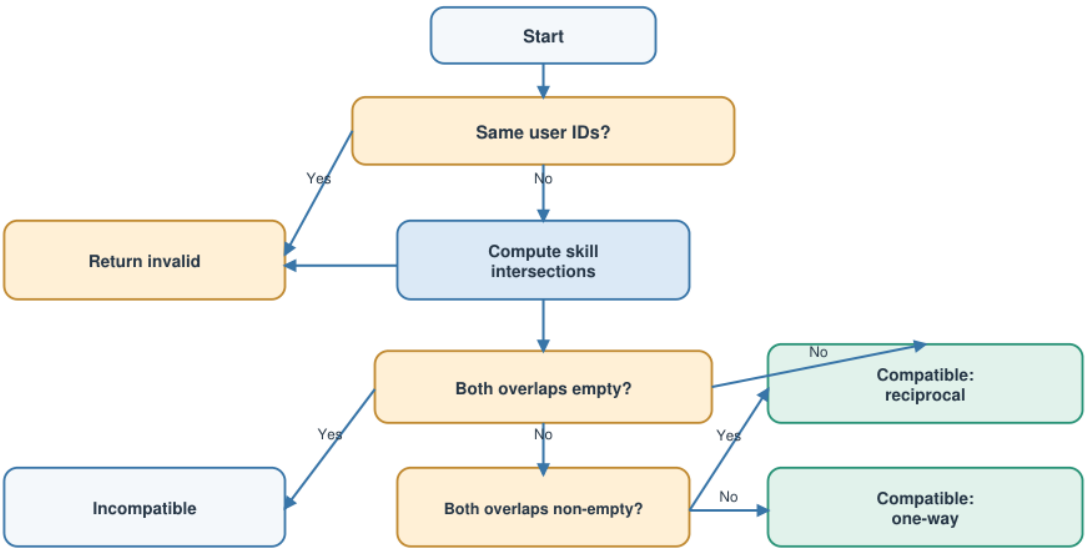


Figure A7. Decision paths used for basis-path test design; cyclomatic complexity is 4.

A8. Basis paths and additional test cases

Path	Condition	Expected result
1	Same user IDs	Invalid
2	Different users; both overlap sets empty	Incompatible
3	Only one directional overlap	Compatible, non-reciprocal
4	Both directional overlaps non-empty	Compatible, reciprocal

Appendix A — Diagram sheet 4